

Burst-Controlled Phantom-Queue Policer in eBPF

FRIEDRICH TSCHIRPKE*, TU Berlin, Germany

CHRISTIAN ORTLEPP*, TU Berlin, Germany

Rate limiting of network flows is an important component required for the enforcement of many network policies such as flow fairness or flow prioritization. The burst-controlled phantom queue policer (BC-PQP) approach addresses common problems with traditional rate limiting approaches.

This project implements the BC-PQP rate limiting approach in eBPF aiming to provide portability, multicore support, application safety, and integration with the Linux networking stack¹. We present two versions of BC-PQP: a simple version with global datastructures and a so-called sharded version, which implements BC-PQP with CPU-local datastructures that are synchronized regularly. Both versions accurately limit rates and control bursts. Additionally, in a non-emulated setup, the simple version has a throughput of up to 13 million packets per second.

We present two fully functional implementations, a portable virtual machine to run and test them in, as well as a discussion of the problems we encountered and ideas for possible future extensions and considerations.

Additional Key Words and Phrases: Linux, XDP, eBPF, BC-PQP, Rate Limiting

1 Introduction

In networks all around the world, we want to enforce different traffic policies such as bandwidth restriction, fairness, and prioritization of users or specific flows. We implement these policies by limiting network traffic rates. For example, to implement flow fairness for n flows, we effectively limit each flow's rate to $1/n$ of the total available bandwidth. As the base of network policy enforcement, we require rate limiting to be efficient and accurate.

Traditionally, there are two common approaches for rate limiting: traffic shapers and traffic policers. Figure 1a shows the functionality of a traffic shaper. Traffic shapers operate by storing all incoming packets in a packet queue or dropping them if the queue is full. From this queue, the shaper dequeues and passes on packets at the rate we want to enforce and thereby ensures that packets never pass any faster than the configured rate limit. Figure 1b shows the functionality of a traffic policer. Traffic policers use a queue of tokens, i.e. a counter, that represents the number of bytes that are allowed to pass. We add tokens to this queue at the rate we want to enforce. Then, we pass an incoming packet whenever there are tokens equal to the packet size or more in the queue. When a packet is passed, we dequeue the respective tokens from the queue. If not enough tokens are present, we drop the incoming packet. Notice, that the most significant difference is in how traffic shapers buffer real packets and traffic policers only store tokens and immediately pass on packets.

Due to the difference in buffering, shapers and policers differ significantly in their properties in operation. On the one hand, the buffering of packets in traffic shapers causes packet delays and may result in cache misses due to the delayed processing of the packets. For traffic policers, packets are most likely still in L1 cache because we process them immediately. Packet buffering also requires additional memory. On the other hand, buffering real packets and

*Both authors contributed equally to this research.

¹<https://github.com/ftschirpke/BC-PQP-eBPF>

Authors' Contact Information: Friedrich Tschirpke, ftschirpke@campus.tu-berlin.de, TU Berlin, Berlin, Germany; Christian Ortlepp, christian.ortlepp@campus.tu-berlin.de, TU Berlin, Berlin, Germany.



This work is licensed under a Creative Commons Attribution 4.0 International License.

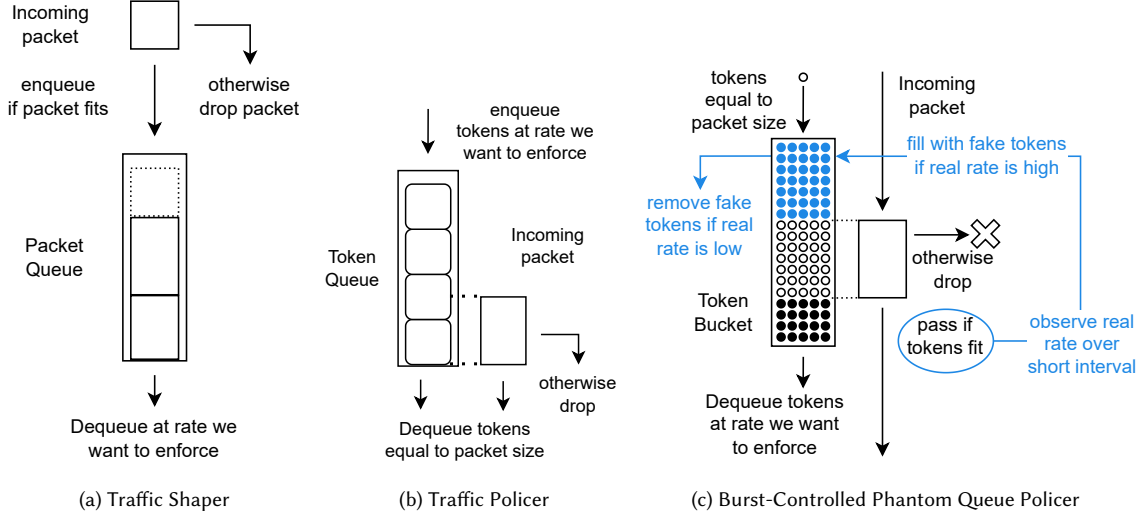


Fig. 1. Schemata showing the functionality of different rate limiting approaches

dequeuing them at the intended rate results in accurate rate enforcement when using traffic shapers. In contrast, traffic policers do not accurately enforce rates for bursty traffic. For bursty traffic, traffic policers may pass on all packets of a burst when the token queue is full and thus temporarily let through packets at a rate that is potentially significantly higher than the rate limit. In conclusion, traffic shapers are accurate with significant overhead and traffic policers are lightweight with significant inaccuracies.

To address the issues of policers, Tahir et al. [1] propose a burst-controlled phantom queue policier (BC-PQP) approach. BC-PQP is a rate limiting approach that uses a token queue, like other policers, with an additional burst control mechanism. Figure 1c shows the functionality of BC-PQP with the differences to a traditional traffic policier highlighted in blue. To prevent large bursts, Tahir et al. propose measuring the rate at which the policier passes on packets over short intervals. If the rate of packets let through by the policier is higher than a chosen threshold, we fill the token queue with fake tokens to limit further traffic to the rate limit we want to enforce. If the rate of packets let through by the policier drops below a chosen threshold, we remove the fake tokens again. This burst control mechanism ensures that rate bursts are limited to the chosen upper threshold chosen when measuring rates over intervals larger or equal in size as the size chosen for burst control.

The goal of this project is to implement BC-PQP with multicore support by implementing it in eBPF. Tahir et al. use Intel’s DPDK for the reference implementation. DPDK is a high-performance framework for packet processing that limits the implementation to a single CPU and is separate from the Linux networking stack. eBPF allows us to integrate BC-PQP with the Linux networking stack, implement multicore support, increase portability, and ensure application safety.

In this project, we implement two fully-functional versions of multicore BC-PQP: a simple and a sharded version, and we provide a reproducible virtual machine that allows to run and test both versions. The simple version is close to the reference implementation with global datastructures shared by the CPUs. Additionally, we present the sharded version, which implements BC-PQP with CPU-local datastructures that are synchronized globally. Both versions

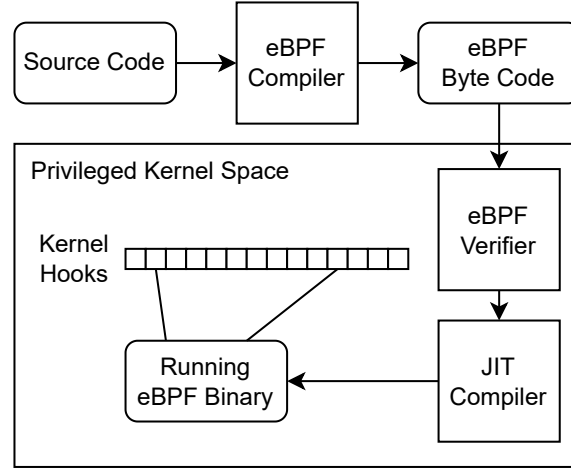


Fig. 2. Workflow of creating and running eBPF programs.

accurately enforce rate limits and limit bursts to chosen thresholds. In experiments on physical servers, we also measure a throughput for our simple version of 3.5 and 13 million packets per second for 4 and 20 CPU cores respectively.

2 Architecture

We choose eBPF and more specifically XDP as the runtime for our BC-PQP implementation. eBPF² is a technology within the Linux kernel which can be used to extend the kernel’s logic at runtime with code provided by the user, without having to change the underlying kernel itself. Conceptually, eBPF is the runtime for programs following the eBPF Instruction Set Architecture (ISA) within the Linux kernel. As the eBPF-ISA is independent of the actual processor architecture a kernel is running on, these programs are portable to other processor architectures. To ensure the correctness of kernels which have loaded eBPF programs, the kernel, more specifically the eBPF verifier, places additional restrictions on the kinds of eBPF programs that are accepted by it. These restrictions include not dereferencing memory which does not belong to the program and making sure that eBPF programs always terminate. eBPF programs are attached to so-called hooks, which determine when and in which context the program is executed. One such hook is XDP, which allows attached programs to redirect, rewrite or drop network packets before they are forwarded to the Linux networking stack.

Figure 2 shows the lifecycle of an eBPF program. We start with a source file, which is usually written in C but can also be any other language that can be compiled to the eBPF-ISA. We compile this source file with a suitable compiler, clang in our case, to eBPF byte code. While these first three steps took place in user space, we now hand our binary over to the kernel. It first checks the correctness of our program using the eBPF verifier. Once it has made sure it is correct, the kernel then JIT-compiles the byte code to the actual processor architecture it is running on. Whether this step is performed depends on the processor architecture, but since it is performance critical, JIT-compilation is implemented for all major architectures. Finally, the compiled program will be attached to its specified kernel hook (or multiple hooks) and is invoked whenever this hook is triggered.

²<https://ebpf.io/>

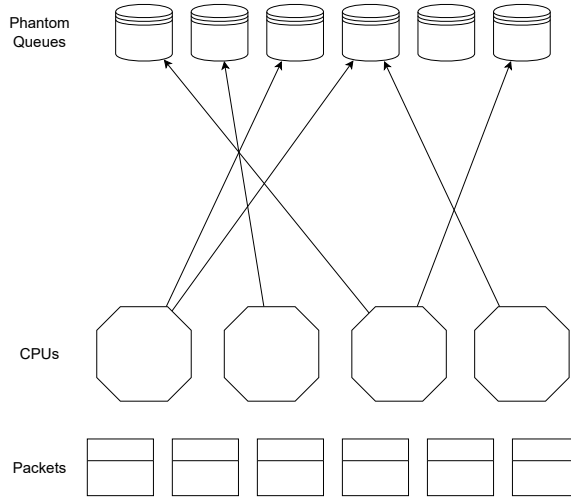


Fig. 3. Architecture overview of the simple implementation.

2.1 Simple BC-PQP Implementation

Figure 3 shows the high-level architecture of our simple implementation. On the bottom, packets arrive at the system and are assigned to one of the physical CPU cores. We make no assumptions about how the packets are distributed among the individual cores. Once a packet has arrived at one of the cores, our program is run and starts by classifying the packet into one of the available phantom queues. This classification is usually done based on packet header data such as addresses or ports. Each phantom queue is represented by a struct in main memory, which holds information such as the queue’s capacity, occupancy and the current throughput. Once the program has determined into which queue the packet belongs, it looks up the respective structure in memory and performs the BC-PQP algorithm on it, updating values in the struct as needed. After successfully performing the algorithm, it now knows whether to drop or forward the packet and will return that verdict to the kernel. For passed packets, we redirect the packet directly to the next destination to avoid unnecessary processing by the kernel networking stack.

It should be noted that this implementation already supports parallel packet processing, as all CPUs can perform the algorithm in parallel. Because of this, all accesses to the shared phantom queue structures need to be synchronized. We use atomic instructions for this synchronization, thus the efficiency of the parallel processing depends on the traffic distribution among the cores: If all CPUs were to process packets belonging to the same phantom queue, they will be competing to access the same data. While we have taken measures in the implementation to allow for such accesses to be able to fail gracefully, i.e. without having to retry or abort the algorithm, this architecture pattern will inadvertently lead to suboptimal scalability in those scenarios. First, atomic instructions are generally less performant compared to their non-synchronized counterparts. Second, accessing the same data from multiple CPUs will prevent that data from being cached effectively in the core-local L1 caches, which is essential to achieving low memory access times. To address these shortcomings, we came up with a second implementation.

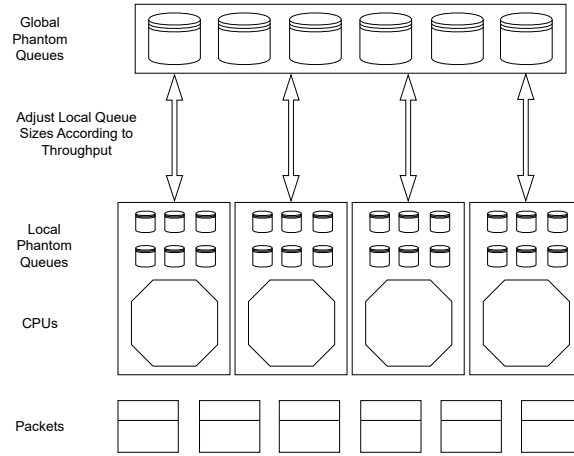


Fig. 4. Architecture overview of the sharded implementation.

2.2 Sharded BC-PQP Implementation

Figure 4 shows the high level architecture of our revised, sharded implementation. The overall design stays the same: Packets arrive at one of the cores, are classified into one of the phantom queues and the BC-PQP algorithm then decides whether to forward or drop the packet, depending on the state of the queue. The major change here is that the CPU cores no longer work on a shared queue structure, but instead each core has its own private, sharded version of each phantom queue. That has the consequence that accesses to the queue structure no longer have to be synchronized, and that the queue structures can reside in core-local caches as they are only accessed by a single CPU core.

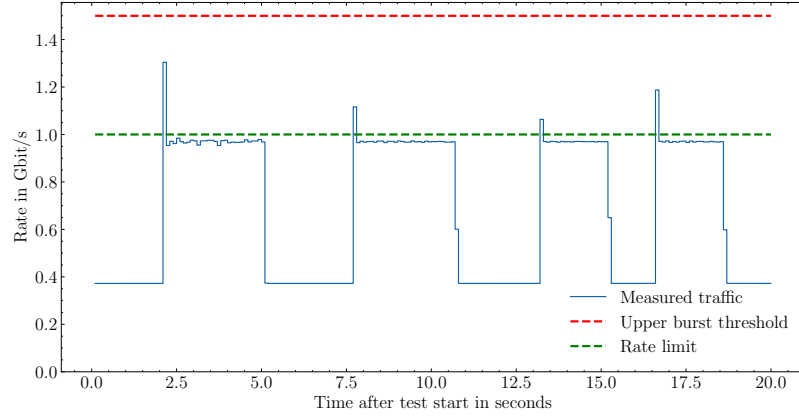
This approach, however, brings a new challenge with it: How can we compose our system-wide rate limit which we want to enforce from the CPU-local sharded rate limits? Or, in other words: How do we have to size the sharded queues so that the sum of the enforced rate-limit over all CPU cores is equal to the desired rate limit? In our implementation, we divide each phantom queue into sharded queues for each physical core. The size of a core’s sharded queue is proportionate to the per-queue throughput processed on the core relative to the total throughput of the respective phantom queue. For example, if a core processes 2 Gbps for a specific phantom queue that receives 4 Gbps of traffic in total, the core’s sharded queue makes up 50% of the phantom queue’s size. We update the sizes of the sharded queues regularly so that we can react to changes in the traffic, and its distribution over the cores.

3 Results

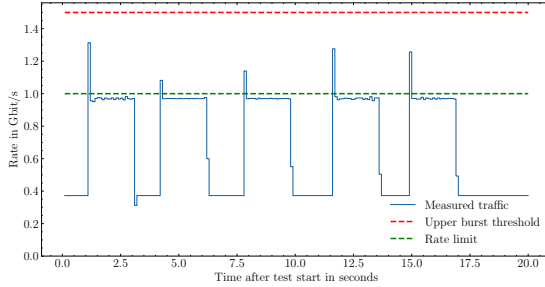
In this section, we present evaluation results for our two implementations. First, we test the rate enforcement and burst control using our virtual machine. Secondly, we run our programs in a physical setup with higher computing power and network bandwidth, and test the rate enforcement and throughput of our programs.

3.1 Qualitative Rate Enforcement in Emulated Environment

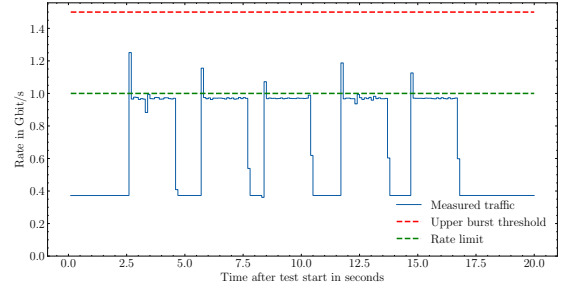
In this experiment, we start our programs in an emulated environment to qualitatively evaluate their functionality. We use a QEMU virtual machine with 4 CPU cores and we are running Alpine Linux 3.22 with a 6.12.38-0-virt Linux kernel. The host machine has an Intel(R) Core(TM) i5-10600KF CPU and 32 GB of RAM. In each run, we configure a rate



(a) Measured throughput for UDP traffic



(b) Measured throughput for TCP CUBIC traffic



(c) Measured throughput for TCP RENO traffic

Fig. 5. Measured throughput of the simple BC-PQP over time for bursty traffic. The dashed lines show the configured rate limit of 1 Gbps in green and the upper burst threshold of 1.5 Gbps in red.

limit of 1 Gbps and send bursty traffic using the network testing tool iperf3. We imitate bursty traffic by sending one continuous flow of 400 Mbps and repeatedly sending a second flow without limit for a few seconds, which effectively gives bursts of up to 7 Gbps. The measuring rate is configured as the iperf3 minimum of 0.1 s.

3.1.1 Simple Version. The simple version of our program accurately enforces the rate limit and robustly handles bursts for both UDP and TCP traffic.

Figure 5a shows the measured traffic for UDP traffic. The graph shows that, whenever there is no burst present, the program lets all packets of the continuous 400 Mbps flow pass. And whenever a burst occurs, there is a spike in the beginning that remains below the upper burst threshold, followed by a measured rate just below 1 Gbps. This indicates, that the program accurately detects the burst, stops it, and then accurately enforces the rate limit of 1 Gbps.

Figure 5b and Figure 5c show the measured traffic for TCP traffic with CUBIC and RENO congestion control respectively. There is no significant qualitative difference between the two TCP results or the UDP result. This indicates, that the program functions independently of the protocol used and that burst control does not seem to clash with congestion control. The continuous TCP flow immediately recovers to the full 400 Mbps after each burst. Important to mention here is that the results may differ when rate limiting a single bursty TCP flow instead of two separate flows.

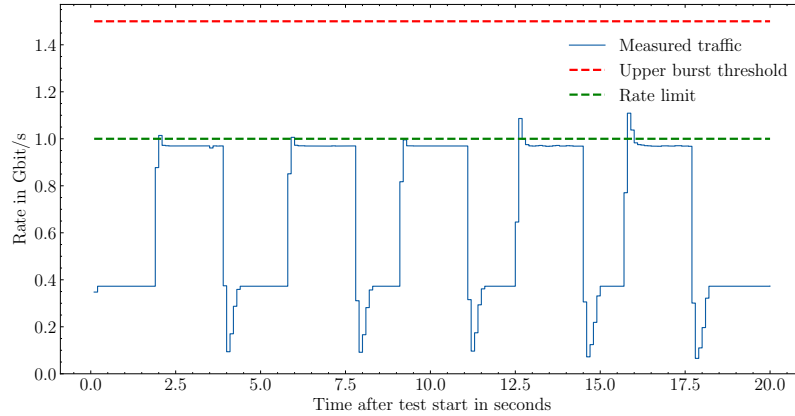


Fig. 6. Measured throughput of the sharded BC-PQP over time for bursty UDP traffic. The dashed lines show the configured rate limit of 1 Gbps in green and the upper burst threshold of 1.5 Gbps in red.

3.1.2 Sharded Version. The sharded version of our program also accurately enforces the rate limit and handles bursts, but by design shows significantly more artifacts whenever the traffic shape changes.

Figure 6 shows the measured traffic for UDP traffic sent through the sharded version of our program. Similarly to the simple version, the sharded version accurately enforces the rate limit of 1 Gbps and limits all burst to below the upper threshold of 1.5 Gbps. However, the figure shows that at the start of the experiment, at the begin of each burst, and after each burst, there are intermediate steps in the graph that do not exist for the simple version.

All of these intermediate steps are expected due to the synchronization approach we use for the sharded version. At the beginning of the experiment, the program needs a bit of time to configure the CPU that receives the traffic for the continuous 400 Mbps flow to have all the capacity. As soon as a burst starts, the program now needs to reconfigure because the second flow arrives at a different CPU, which means that the first CPU now should have a smaller share of the total capacity and the second CPU, which receives the large burst of traffic, needs to be configured as having a large share of the total capacity. And as soon as the burst ends again, this needs to be configured back to the state before the burst, which, due to the rolling average, requires a bit of time. Thus, these artifacts are expected causes of the design choices and should not be seen as errors but rather as part of the tradeoff when choosing between the simple and the sharded version.

3.2 Rate Enforcement and Stress Testing on Physical Hardware

In the following experiments, we use two physical servers. One server generates UDP traffic, sends it to the second server, receives the traffic again, and measures the throughput. The second server has our BC-PQP program loaded and thus performs rate limiting on the received traffic. Both servers are equipped with an Intel(R) Xeon(R) Gold 6209U CPU@2.10GHz with 20 physical cores and hyperthreading capabilities, 192 GB of RAM, and two 25G NICs (Intel XXV710 for 25 GbE SFP28 (rev02)). They run Ubuntu 22.04.4 LTS with a 6.5.0-35-generic Linux kernel.

3.2.1 Simple Version: Rate Enforcement. In this first experiment, we test the rate enforcement of our simple version in a real-word setup by generating 25 Gbps of traffic and measuring the average throughput for different configured rate

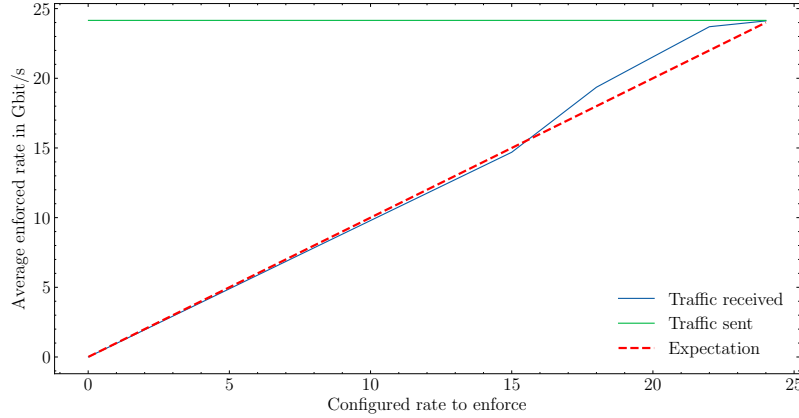


Fig. 7. Measured average throughput of the simple BC-PQP for different rate limits.

limits. For accurate rate enforcement, we expect the average throughput to always be equal or slightly lower than the configured rate limit.

Figure 7 shows that simple BC-PQP enforces the rate limit accurately for rate limits up to approximately 15 Gbps and fails for higher rates. For low rates, our expectation holds. The average throughput is always below the rate limit, indicating that the rate limit is enforced accurately. However, for rate limits of approximately 18 Gbps and higher, the average throughput is higher than the rate limit. Thus, the rate limiting fails for these high rates. This indicates some problems in our implementation that only trigger or become significant at high rates.

3.2.2 Simple Version: Throughput. In this second experiment, we test the throughput of the simple version of our program by measuring the average throughput when generating approximately 25 Gbps of traffic and setting the rate limit higher than that for different numbers of CPU cores. We do this for MTU-sized packets and 64-byte packets to stress test the program and measure its maximum throughput. For 64-byte packets, the hardware is unable to generate the full 25 Gbps and instead only generates approximately 11.5 Gbps.

Figure 8 shows that simple BC-PQP has a throughput of at least 24 Gbps of MTU-sized packets regardless of core count and a throughput scaling up to 6 Gbps of 64-byte packets for 20 CPU cores. Figure 8b shows that for MTU-sized packets, the difference between the traffic sent and the traffic received is insignificantly small, indicating that our program is not at full capacity and has a higher throughput than the 24 Gbps sent here. Figure 8a shows the results for traffic made up of 64-byte packets. Here, the measured throughput scales up non-linearly from 1.8 Gbps when using 4 CPU cores to up to 6.3 Gbps when using 20 CPU cores. This shows that, as expected, the throughput scales up with the number of CPU cores. Additionally, we see that hyperthreading does not seem to have any significant effect on the program’s throughput as the throughput is approximately 6.3 Gbps from all numbers of CPU cores from the physical limit of 20 CPU cores up to the limit with hyperthreading of 40 CPU cores.

This shows that even our simple version has good performance on real-world systems. The throughput rate for 64-byte packets gives us an estimated packet throughput of our program of approximately 13 million packets per second when using 20 CPU cores, and approximately 3.5 million packets per second when using 4 CPU cores. As packets in real traffic are significantly larger, closer to MTU than 64 bytes, we can expect our program to handle large rates of real-world traffic.

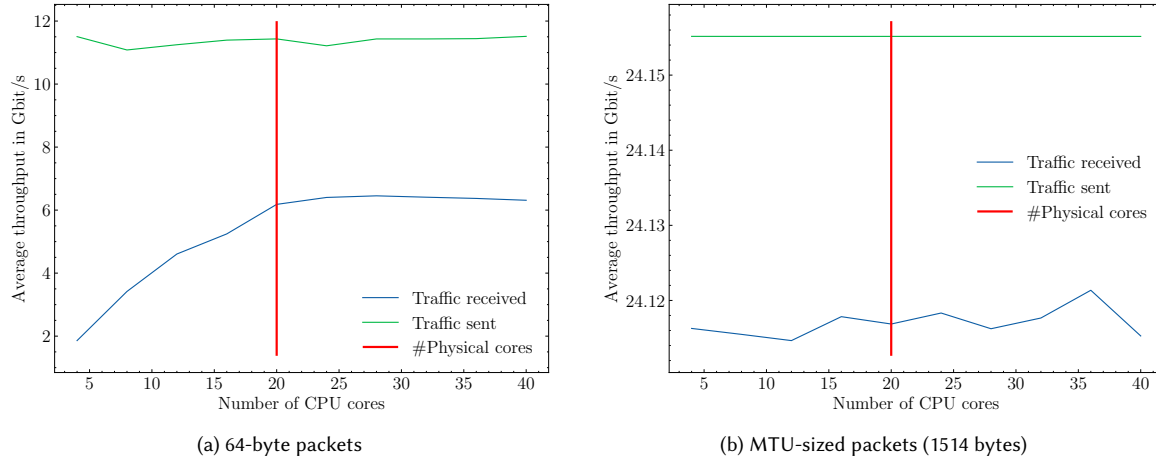


Fig. 8. Average throughput of the simple version for different numbers of CPU cores.

A baseline run also shows that much of the overhead does not stem from our implementation. We run the identical experiment as described above for a baseline program that only accepts and forwards packets but does not do any of the checks and calculations required for rate limiting or burst control. Thus, this baseline program shows the overhead of simply having an XPD program loaded and operate on every incoming packet. The experiment shows that the baseline program has a throughput of approximately 7.9 Gbps of 64-byte packets when using 20 CPU cores. Comparing the baseline throughput of 7.9 Gbps with the sent traffic of approximately 11.5 Gbps and the throughput of 6.3 Gbps of our simple BC-PQP program, we see that much of the overhead is already present for the baseline program. Possible sources of this overhead are the XDP/eBPF subsystem and the high packet processing overhead for small packets in general.

3.2.3 Sharded Version. Due to time pressure in the last weeks of the project, we were unable to successfully test the sharded version on the physical servers. With the synchronization approach of the sharded version comes increased complexity and per-cpu datastructures, which both need to be configured correctly to work. Additionally, we do not control all variables for the physical setup like we do for the virtual machine. Thus, we were unable to configure our implementation correctly in time to perform the experiments.

4 Implementation Challenges

Firstly, navigating the eBPF ecosystem can be challenging, even though the technology is quite established at this point. What we mean by this is that large parts of the workflows are not at all or only sparsely documented, or that certain tools are hard to work with. The compilation process from C to eBPF byte code, for example, is prone to breakage in clang. For example, most programs do not pass the verifier if they were not compiled with the optimization level 2 (-O2), which is not documented except in an LWN article³. Further, atomic operation built-ins that according to the official documentation return a value, in reality do not. Except for an open GitHub issue in the LLVM repo, this is not documented⁴. While the latter at least leads to an immediate compilation error, the former compiles successfully but is rejected by the verifier (without a usable error message). In general, working with the verifier, especially in

³<https://lwn.net/Articles/946254/> (retrieved 22-09-2025)

⁴<https://docs.ebpf.io/linux/concepts/concurrency/https://github.com/llvm/llvm-project/issues/35921> (retrieved 22-09-2025)

the beginning, is challenging. It provides very few easily digestible messages, and instead simply prints the formal reasoning that caused the rejection of a certain instruction. This often does not help developers that are unexperienced with the eBPF-ISA and the verifier’s reasoning process.

Secondly, to develop eBPF-BC-PQP we needed a reproducible testing and development setup. Tools like docker or podman are insufficient for our purposes, as they do not virtualize the kernel. We, however, also did not want to just commit an entire virtual machine image to the repository, as such an image would be a “black box”, where changes are hard to track. In the end, we managed to use docker/podman to first construct a virtual file system for a VM, export this file system and convert it to a virtual machine image that is understood by virt-install/QEMU. This way, we track all changes in a simple Dockerfile, and we have a completely virtualized environment (that includes a kernel).

Thirdly, writing low-level kernel programs is in its nature challenging in some aspects. Because these programs tend to be performance critical, we cannot afford to build as many abstraction layers as we would when only prioritizing code quality and code readability. That results in a lot of side effects and state being mutated from different places, which makes the code easy to break. This is exacerbated by the fact that our programs are inherently parallel, and it is therefore easy to introduce race conditions. This is especially true for our sharded implementation, where each CPU core acts autonomously and is cut off from the rest of the system, making this program (in some aspects) behave like a distributed system, with all the difficulties that such a system comes with.

Finally, testing and asserting that our implementation behaved correctly proved to be challenging as well. While there are many different network traffic generation tools out there, none of them really satisfied all of our requirements: Flent comes with an impressive amount of testing suites, but due to its outdated base it is hard to install, and the testing suites are fixed e.g. we were unable to measure UDP throughput at sufficient rates. iperf3 on the other hand is quite easy to work with but supports only coarse grained measurements (0.1 second intervals) and does not implement more complex test scenarios out of the box. In the end, we settled on using iperf3 for our experiments in the emulated environment and MoonGen for the experiments with the physical servers.

5 Lessons Learned

Observability proved to be a major pain-point in the end. With time, our implementation got more and more complex, yet our tooling to debug and observe the program did not mature with it. Our main method for testing the program and finding bugs was still to run load tests, generate graphs and investigate if something did not look as expected. We did also build some logging instrumentation, this however proved less helpful than we hoped because it generated too many messages for a human to parse, even when enforcing relatively small rates. Going forward, it would definitely also make sense to add unit tests for as many parts of the program as possible. We chose not to do so because it would have meant significant additional development overhead, and at least in the beginning it would not have had many benefits because we were rapidly iterating over different designs, which would have likely broken any unit tests too⁵.

Something that took us a bit more by surprise was just how much our results depended on the testing environment, e.g. whether the kernel was virtualized or with which exact rates we were testing. Some issues, for example, only appeared when enforcing rates above 15 Gbps (see Section 3), which is a rate we could not achieve in our virtualized environment. While our QEMU-based development and testing setup was great for rapidly testing out new changes and seeing results immediately, a successful test there does not guarantee that the results translate to a non-virtualized production server. To catch such issues earlier in the future, we as developers would have needed more immediate access

⁵https://docs.ebpf.io/ebpf-library/libbpf/userspace/bpf_prog_test_run_opts/ (retrieved 22-09-2025)

to such non-virtualized environments for testing, ideally we would integrate such tests in a continuous integration pipeline.

6 Future Work

As we already touched on briefly in Section 4, our ability to test the implementations in a non-virtualized environment and with high rates was very limited. Doing so, and to investigate and fix any issues that come up would be necessary to make the implementations usable and correct in more environments. Another possible quality improvement would be to investigate how the synchronization artifacts in the sharded version can be reduced, for example by no longer performing the synchronization only regularly but also doing so “on demand”.

Depending on the use-case, BC-PQP may be used with many queues, for example if a telecommunications provider wishes to abstract each customer with a distinct queue. In such cases, not all queues might fit into the core-private caches of the CPUs (this is only relevant for the sharded implementation). It would thus make sense to put frequently accessed queues next to each other in memory, so that they will be more likely to end up being cached.

The sharded implementation of BC-PQP also poses some interesting theoretical questions that were not covered by the original BC-PQP paper [1]. These stem from the fact that in original BC-PQP, queues were considered to be static and never changed their capacity. In the sharded implementation, however, we need to divide the logical phantom queues into separate sharded phantom queues in order to approximate the packet distribution among individual CPU cores. Because this distribution can change over time, we also need to adjust the queue sizes. This begs the question of how state needs to be transferred between the sharded queues when such a resizing happens in order to achieve the rate bounds which were calculated by the original BC-PQP paper, or if new bounds are needed for this implementation.

The BC-PQP rate limiter has some configuration options for which the optimal configuration is not immediately obvious, such as the rate-to-queue-capacity ratio and the size of the burst interval and the synchronization interval (the latter only applies to the sharded implementation). Investigating which values are optimal for different environments, how they can be found, and what tradeoffs exist are areas of future work.

Finally, in our work we assumed that the rules by which traffic is classified, as well as the rates that should be applied, are given and fixed. In practice, it is more likely that a user would like to apply a certain traffic policy to their system, for example fairness between certain kinds of flows or prioritization of one flow over others. An area of future work would therefore be to write a user space component that can translate such policies into concrete rate limits which are then understood by BC-PQP, as well as managing and adapting this configuration as needed.

7 Summary

In this project, we implemented the BC-PQP approach in eBPF with multicore support in two ways: the simple version and the sharded version. Both versions accurately enforce rate limits and control burst when run in the portable virtual machine we provide. Experiments with physical servers show that the simple version also accurately enforces rate limits for rates up to 15 Gbps and has a throughput of approximately 13 million packets per second when using 20 CPU cores.

We encountered multiple problems along the way. The eBPF environment caused us problems in the initial setup and continuous compilation problems. The complexity of multicore systems led to many unexpected implementation issues. And the environment around network testing tools left us restricted in what we were able to test.

While overcoming the problems, we learned a lot and now know what we would consider doing differently. After developing in an emulated environment, we would next time consider the disadvantages of doing so more closely. We

also would next time start with building a more solid debugging and testing setup before working on implementation features.

Despite the challenges, we produced two multicore implementations of BC-PQP in eBPF that work accurately in the portable emulated environment we provide as well. Experiments on physical servers show promise for use in real-world setups in regard to correctness and scalability.

Acknowledgments

We would like to thank our supervisor, Jonas Köppeler, for taking the time to sit down with us every week, for providing invaluable feedback and advise, and for providing us with the ability to evaluate our implementation on physical hardware.

References

- [1] Ammar Tahir, Prateesh Goyal, Ilias Marinos, Mike Evans, and Radhika Mittal. 2024. Efficient Policy-Rich Rate Enforcement with Phantom Queues. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney NSW Australia, 2024-08-04). ACM, 1000–1013. doi:10.1145/3651890.3672267